The Living Project

Whitepaper

A model-agnostic framework for AI-assisted project work

Version 3.0.3 | @the-living-project/the-living-project-cli | Author: Chamal Abeysekera | CC BY 4.0

Executive summary

The Living Project is a model-agnostic framework and installable CLI for running AI-assisted project work with stronger context, better iteration discipline, and safer release practices. It combines a one-line public installer, a structured local workspace, an optional Codex skill, and a release process that supports versioned upgrades over time.

The framework treats projects as living systems. Seeds are planted with clear problem definitions. Context is loaded as fertilizer before generation begins. Outputs grow through directed iteration, get pruned through structured critique, and when directions fail, they are composted -- broken down into validated learnings that feed the next cycle. The workspace itself becomes the durable operating model.

The current live package is @the-living-project/the-living-project-cli, installable with:

```
npx -y @the-living-project/the-living-project-cli init
```

The problem

Most AI-assisted work fails for predictable reasons:

- Teams start with deliverables instead of the real problem. The first prompt is "write me a strategy document" when it should be "help me understand what problem we are actually solving."

Context is incomplete, fragmented, or never written down. The AI gets a thin slice of what matters and fills the gaps with generic best practices.

- First drafts are treated as finished work. A single generation pass is shipped without structured critique, review, or iteration.

Scope drift is noticed too late. Requirements shift incrementally until the project is solving a different problem than the one it started with.

- Abandoned efforts are not mined for reusable insight. When a direction fails, the learnings die with it instead of feeding the next attempt.

Every new project starts from scratch. There is no reusable system for starting, iterating, reviewing, and completing AI-assisted work.

The Living Project addresses these issues by turning the workspace itself into a durable, upgradeable operating model that any AI tool can read and write to.

Framework design

The Living Project uses a seven-phase structure organized into a core loop and three strategic phases.

Core loop: SEED -> NOURISH -> GROW -> PRUNE -> repeat until the output is ready to ship or hand off.

Strategic phases: REPOT | COMPOST | CULTIVATE -- invoked when scope changes, direction fails, or multiple efforts need portfolio-level management.

Phase overview

Phase	Purpose	Key technique	Output location
1. SEED	Define the real problem and root need.	5 Whys analysis + 5W framework (Who, What, Where, When, Why).	.living-project\seeds\
2. NOURISH	Load every piece of relevant context before generation.	Context inventory, contradiction detection, gap analysis.	.living-project\context\
3. GROW	Generate directional drafts, options, and candidate outputs.	Variant generation (conservative, ambitious, unconventional). Rapid iteration.	Working files
4. PRUNE	Critique, improve, reduce risk, and tighten quality.	Seed-restated critique, 1-10 scoring, robustness testing.	Refined output
5. REPOT	Restructure scope or architecture when the container is too small.	Constrain vs. expand vs. split analysis with effort and risk estimates.	.living-project\seeds\ (updated)
6. COMPOST	Extract learnings and restart with stronger insight.	Validated/invalidated assumptions, reusable component extraction, post-mortem.	.living-project\compost\
7. CULTIVATE	Manage multiple active projects as a portfolio.	Weekly review: progress comparison, risk flagging, cross-pollination.	.living-project\cultivate-log.md

Phase 1: SEED -- Plant the Question

Every project begins with a question, not an answer. The Seed phase uses the 5 Whys technique to move past symptoms to root causes, then structures the finding into a one-paragraph seed statement answering: Who is this for? What does it solve? Where does it live? When is it needed? Why does it matter now? The seed is saved

to .living-project\seeds\ and becomes the measuring stick for every subsequent phase. Practitioners also identify at least two assumptions and two blind spots before proceeding.

Phase 2: NOURISH -- Context Is Fertilizer

This is the phase most teams skip, and it is the single biggest determinant of output quality. Nourish loads the AI workspace with prior art, stakeholder notes, technical constraints, data, competitive analysis, and institutional knowledge. The result is a structured context brief saved to .living-project\context\. The AI validates completeness by flagging contradictions in source material and identifying what context is still missing.

Phase 3: GROW -- From What Exists to What Should Exist

Grow bridges the gap between current state (documented in Nourish) and desired state (defined in Seed). Generation is directional: every prompt includes context, outcome, and constraints. The framework biases toward quantity over perfection -- practitioners generate conservative, ambitious, and unconventional variants, then iterate rapidly. No first draft exits Grow without at least two rounds of refinement.

Phase 4: PRUNE -- Check Your Work, Make Improvements

AI generates plausible content, not necessarily correct content. Prune turns the AI against its own work. The seed statement is restated as the measuring stick. The AI identifies gaps, redundancies, logical errors, and risks, then scores the output 1-10 with justification and repairs what it found. Robustness testing validates the output against known past failures. Every element must earn its place.

Phase 5: REPOT -- Does It Need a Bigger Pot?

Repot evaluates whether a project has outgrown its scope, resources, or architecture. This is deliberate scope evolution, not scope creep. The framework distinguishes Repot from Compost using clear signals: if the core architecture is sound and assumptions are still valid, Repot. If assumptions are invalidated or the team is spending more time fixing than building, Compost instead. Repot produces a constrain, expand, or split decision with updated seed statements.

Phase 6: COMPOST -- Decompose the Old, Fertilize the New

Composting is not failure. It is the productive act of breaking down a completed or stalled effort so its nutrients feed the next thing grown. The Compost phase extracts validated assumptions, invalidated assumptions, reusable components, and key learnings. It produces a post-mortem saved to .living-project\compost\ and a stronger new seed brief that incorporates everything learned.

Phase 7: CULTIVATE -- Tend to Several at Once

Cultivate provides portfolio awareness across active project threads. On a weekly cadence, practitioners step back to compare progress, flag risks and dependencies, identify cross-pollination opportunities between projects, and decide which efforts need Repot or Compost decisions. The portfolio view is maintained in .living-project\cultivate-log.md.

Multi-agent orchestration

Each phase includes a documented multi-agent pattern for teams using AI tools that support parallel agent execution:

Phase	Agent strategy
SEED	Agent A runs 5 Whys. Agent B researches prior solutions. Agent C challenges the problem statement.
NOURISH	Agent A summarizes internal docs. Agent B researches external landscape. Agent C maps stakeholder positions.
GROW	Agent A generates conservative approach. Agent B generates ambitious approach. Agent C generates unconventional approach.
PRUNE	Agent A critiques for technical accuracy. Agent B critiques for audience fit and clarity.
REPOT	Agent A models the constrained path. Agent B models the expanded path.
COMPOST	Agent A performs post-mortem extraction. Agent B researches landscape changes since project start.
CULTIVATE	Each project runs its own agent thread. Cultivate is where the gardener reviews across all threads.

Product architecture

The implementation has four layers:

1. Public npm package -- distributed through npm as @the-living-project/the-living-project-cli. Installable via npx, pnpm dlx, bunx, or yarn dlx. Requires Node.js 18 or later.

CLI runtime -- a single Node.js executable that handles init (create workspace), upgrade (refresh managed files), and doctor (health check). Supports flags for workspace naming, installation location, skill root override, and client adapter control.

2. Workspace payload -- creates a project folder with .living-project\ containing seeds, context, compost, phases, cultivate-log, and QUICKSTART.md. A visible START-HERE.md sits at the workspace root.

Optional Codex skill -- installs a local living-project skill into ~/.codex/skills/ when Codex is detected. The skill provides automatic phase routing based on user intent. When Codex is not present, the workspace remains fully usable through the universal prompt in START-HERE.md.

Workspace contract

The installer creates a workspace that behaves as the durable memory layer for the project. Files are divided into managed (framework-owned, refreshed on upgrade) and user-owned (never overwritten).

Path	Owner	Purpose
START-HERE.md	Managed	Visible first-run handoff with universal prompt and Codex shortcut.
.living-project\QUICKSTART.md	Managed	Zero-training start guide.
.living-project\phases\	Managed	Seven phase guidance files read by the AI assistant.
.living-project\.gitignore	Managed	Excludes private context and compost from version control.
.living-project\seeds\	User	Seed statements for each project or sub-project.
.living-project\context\	User	Context briefs, stakeholder inputs, and working memory.
.living-project\compost\	User	Post-mortems, validated learnings, and reusable components.
.living-project\cultivate-log.md	User	Weekly portfolio review journal. Created once, never overwritten.

Installation experience

The design goal is zero-training startup.

3. Run one command: npx -y @the-living-project/the-living-project-cli init

Open the generated workspace.

4. Paste the universal prompt from START-HERE.md into any AI tool.

The AI reads the workspace, detects the current phase, and guides the user forward.

This minimizes onboarding burden and makes the framework usable by people who do not already know the vocabulary. The universal prompt works with any capable AI tool. When the Codex skill is installed, users can also invoke the framework with a shorthand trigger.

CLI reference

Command / flag	Description
init [workspace-name]	Install the framework. Creates workspace folder, skill, and all framework files.
upgrade [workspace-name]	Refresh managed files and upgrade the installed skill. Preserves all user work.
doctor	Inspect workspace and skill health. Reports installed versions and missing components.
help	Show full help text with all available flags and examples.
--here	Install into the current directory instead of creating a new folder.
--root <path>	Root directory for workspace creation.
--workspace-name <name>	Explicit workspace name (alternative to positional argument).
--skills-root <path>	Override the default Codex skills directory (~/.codex/skills/).
--client <auto|codex|none>	Control client adapter: auto detects Codex, codex forces install, none skips.
--skip-skill-install	Do not install the Codex skill.
--skip-workspace-install	Do not create or update workspace files.
--no-backup	Skip backups before replacing managed files during upgrade.
--json	Output doctor results as machine-readable JSON.

Model-agnostic strategy

The Living Project is not tied to a single model vendor. It stays portable by separating framework content, workspace structure, and optional client-specific integrations.

The same workspace can be used with:

- Codex, using the installed local skill for automatic phase routing.

Claude, ChatGPT, Gemini, or any other AI tool, using the universal prompt in START-HERE.md.

- Custom integrations, by reading the phase files and workspace structure directly.

.

-

- This keeps the method portable while still allowing richer local experiences where supported.

Upgrade model

The project supports versioned distribution and repeatable upgrades. This is essential for enterprise use because the framework itself can evolve while active project material remains intact.

How upgrades work:

5. Run: npx -y @the-living-project/the-living-project-cli upgrade

Managed files (phases, QUICKSTART.md, .gitignore, START-HERE.md) are backed up, then replaced with the latest version.

6. User work in seeds, context, compost, and cultivate-log.md is never touched.

The Codex skill is replaced with the new version after backing up the previous installation.

7. An install-manifest.json records the version, timestamp, and previous version for traceability.

8.

9.

Release infrastructure

- Source of truth: GitHub (The-Living-Project/the-living-project-cli).

Distribution channel: npm (@the-living-project/the-living-project-cli).

- Release automation: GitHub Actions with npm trusted publishing via OIDC.

CI validation: package behavior is tested on main and pull requests.

- Tag format: living-project-vX.Y.Z.

Tagged releases publish automatically through the release workflow.

Release history

Version	Summary
3.0.0	First public npm release. Full workspace framework, CLI runtime, and Codex skill.
3.0.1	Improved executable mapping for npm launch behavior.
3.0.2	Simplified direct npx usage.
3.0.3	Fixed Windows packaging behavior for workspace template .gitignore. Verified one-line install flow.

Key principles

1. Context is fertilizer. Always front-load context before generation.

Never start with "write me a..." Start with the problem, not the deliverable.

2. Use parallel agents. Multiple perspectives beat sequential refinement.

Restate the seed when pruning. The agent needs the measuring stick.

3. Composting is productive. Nothing is wasted -- learnings feed the next cycle.

The workspace is the system. Not just a prompt pack -- it is an operational wrapper.

Current state

As of version 3.0.3:

- The GitHub repository is live and is the source of truth for all changes.

The public npm package is published and installable worldwide.

- The one-line installer is tested and working across Windows, macOS, and Linux.

Trusted publishing is configured through GitHub Actions for secure, automated releases.

- CI validates package behavior on main and pull requests.

The workspace is usable with any AI tool through the universal prompt.

- The Codex skill is auto-installed when Codex is detected, providing automatic phase routing.

.

-

Why this matters

The Living Project is not just a prompt pack or template bundle. It is an operational wrapper around AI-assisted project work. Its main value is not only better outputs, but a better system for producing, revising, and sustaining those outputs across time.

In practice, it gives teams:

- A repeatable way to start projects -- from root cause, not from a requested deliverable.

A consistent method for loading context -- so AI generation is specific, not generic.

- A structured review habit -- so outputs are critiqued before they are shipped.

A productive way to handle failure -- composting extracts value from every stalled direction.

- Portfolio awareness -- so multiple projects are managed as a garden, not a pile.

An upgradeable framework -- that evolves over time without destroying user work.

- A portable system -- that moves across AI tools without vendor lock-in.

.

-

Conclusion

The Living Project has moved from concept to deployable product. It now exists as a public npm package, a working CLI, a reusable workspace framework, and a release-managed system that can be improved over time without breaking user adoption. The seven-phase model provides the discipline most AI-assisted work is missing: start with the real problem, load sufficient context, generate with direction, critique before shipping, and recycle learnings when directions change.